Distributed Edge AI Cluster — Complete Project Flow

Step-by-step guide from hardware procurement to a fully operational AI research cluster

■ START HERE — Phases 1, 2 & 3

1 PHASE 1 — Hardware Setup

Get the physical components ready

1 Buy the Components

- 4 × StarFive VisionFive 2 (RISC-V single-board computers)
- 1 × D-Link DGS-F1010P-E Gigabit Switch (provides power via PoE)
- 4 × 64 GB MicroSD cards (Class 10 / UHS-1 speed)
- 4 × USB-C power supplies (5V, 3A minimum per board)

2 Physically Assemble the Cluster

- Connect all 4 boards to the switch using Ethernet cables
- Topology: Star — switch in centre, all boards connect to it
- 1 board = Master Node | 3 boards = Worker Nodes
- Check that the green link LEDs light up on the switch ports

3 Set the Boot Mode Switch

- Each board has a tiny DIP switch near the HDMI port
- Set Pin 1 = ON and Pin 2 = ON → this tells the board to boot from MicroSD
- Other modes: eMMC storage (Pin1=ON, Pin2=OFF), QSPI Flash (both OFF)
- Apply power — the board will start loading the OS from your MicroSD card

4 Confirm Hardware Specs Are Working

- CPU: SiFive U74 quad-core @ 1.5 GHz (RISC-V 64-bit)
- RAM: 8 GB per board = 32 GB total across the cluster
- Network: 20 Gbps switch backplane — fast inter-node communication
- All 4 nodes should boot and show a login prompt

Next phase

2 PHASE 2 — OS & Storage

Install the operating system on every board

5 Flash the OS Image onto MicroSD

- Download the official Debian 13 (Trixie) image for RISC-V from StarFive
- Insert MicroSD into your laptop. Find it with: lsblk (Linux) or diskutil list (Mac)
- Flash it: dd if=debian.img of=/dev/sdX bs=1M (replace sdX with your card)
- Wait for it to finish — this copies the whole OS to the card

6 Expand the Storage Partition

■ IMPORTANT

- Problem: the image only uses ~3.8 GB, but your card is 64 GB — 60 GB is wasted!
- Fix with fdisk: delete the unused ghost partitions (p5, p6)
- Recreate partition 4 stretching all the way to end of card
- Run resize2fs to stretch the filesystem → you now have ~59.5 GB free

7 Configure Software Repositories

- Edit /etc/apt/sources.list on all 4 nodes to point to Debian Trixie repos
- Repos give your board access to thousands of software packages
- Run: sudo apt update && sudo apt dist-upgrade to get latest packages
- Do this on all 4 nodes — master first, then workers

8 Set Language & Locale

- Run: sudo locale-gen en_US.UTF-8 on every node
- This stops annoying 'locale not set' warnings when you open a terminal
- Small step but saves confusion during long SSH sessions
- Repeat on all 4 nodes — takes under a minute each

Next phase

3 PHASE 3 — Network & SSH

Make all nodes talk to each other securely

9 Assign Fixed IP Addresses

- Without fixed IPs, addresses change every reboot — chaos ensues
- Master internal: 192.168.50.10 (talks to all workers)
- Worker 1: 192.168.50.11 | Worker 2: .12 | Worker 3: .13
- Use nmcli (NetworkManager) to set these permanently

10 Set Up Passwordless SSH

- SSH lets you control one computer from another over the network
- On Master: run ssh-keygen to create a unique security key (like a digital ID card)
- Copy that key to each worker: ssh-copy-id debian@worker1 (repeat for 2, 3)
- Now Master can connect instantly without typing a password every time

11 Lock Down Worker Security

- Disable password login on workers — keys only (much harder to hack)
- Edit /etc/ssh/sshd_config: set PasswordAuthentication no
- Also set PermitRootLogin no — nobody should SSH in as root
- Restart SSH service: sudo systemctl restart ssh

12 Set Up Internet Sharing (NAT)

- Workers have no direct internet connection — only the Master does
- Enable IP forwarding on Master so it can relay traffic: sysctl net.ipv4.ip_forward=1
- Add iptables rule: traffic from workers exits through Master's WAN port
- Workers can now ping google.com through the Master as a gateway

START HERE (Phases 1–3) — CONTINUE HERE → (Phases 4+5 + Completion)

■ Phases 4 & 5 ■

4 PHASE 4 — AI Toolchain

Install & compile the machine learning software stack

13 Install Compilers & Build Tools

- Compilers turn human-readable code into instructions the CPU can run
- Install: gcc-13, clang-19, cmake, ninja-build, git on ALL nodes
- Also install Python 3 dev libraries — AI frameworks need these
- Verify same versions across cluster: gcc --version on each node

14 Build the IREE Runtime (AI Model Runner)

COMPILE STEP

- IREE converts AI models into optimised machine code the RISC-V CPU understands
- Clone the IREE source from GitHub (takes a while — it's a large project)
- Build with cmake + ninja: turns the source code into a runnable program
- Install iree-run-module to /usr/local/bin/ on each worker node

15 Compile TensorFlow Lite 2.15

COMPILE STEP

- TensorFlow Lite runs AI models efficiently on small/embedded devices
- Clone TF v2.15.0 from GitHub. RISC-V needs a special flag: XNNPACK=OFF
- Compile with: make -j4 — uses all 4 CPU cores to speed up the build
- Install the shared library so all programs on the node can use it

16 Add the Google Coral EdgeTPU Accelerator

7.9× SPEEDUP!

- EdgeTPU is a USB AI chip — like a turbocharger plugged into your USB port
- No pre-built driver exists for RISC-V, so we compile libedgetpu from source
- Set USB permissions (udev rules) so the board can talk to the EdgeTPU chip
- Result: AI inference goes from 24.5 ms → 3.1 ms (7.9× faster!)

Next phase

5 PHASE 5 — Final Checks

Verify every layer before running experiments

17 Check 1: Storage OK?

- Run df -h / on all nodes — should show ~55 GB free
- Run sudo apt update — should complete with no key errors

18 Check 2: Network OK?

- From any worker run: ping -c 3 master → should get 0% packet loss
- Run ip route show → should show Master IP as the default gateway

19 Check 3: SSH OK?

- From Master run: ssh worker1 then ssh worker2 then ssh worker3
- Should connect instantly — no password prompt, no waiting

20 Check 4: Internet Sharing OK?

- From a worker run: ping -c 3 8.8.8.8 → 0% loss = internet works via Master
- Also run: ping -c 3 google.com → tests DNS resolution end-to-end

21 Check 5: AI Runtime OK?

- Run: iree-run-module --help → should print usage info cleanly
- Run a test image through MobileNet V2 — should infer in ~3.1 ms with EdgeTPU

✓ CLUSTER FULLY OPERATIONAL

All 5 checks passed — Ready for Edge AI Research

32 GB RAM • 20 Gbps Interconnect • 322 FPS EdgeTPU Inference

Platform: StarFive VisionFive 2 (RISC-V) • OS: Debian 13 (Trixie) • Cluster: 1 Master + 3 Workers • AI: IREE/MLIR + TensorFlow Lite + Google Coral EdgeTPU